DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Dynamic Memory Allocation

- Preliminaries
- Dynamic Memory Allocation

More Types

`size_t`:

- A type alias for some unsigned integer type.
- Can store the size of the theoretically largest possible object.
- The return type of `sizeof`, `alignof`, and `offsetof`.

`ptrdiff_t`:

- A type alias for some signed integer type.
- The result type when subtracting two pointers.
- The type used in array indexing.

`void*`:

- A pointer to any type.
- Is implicitly convertible to any other pointer type.

---

https://en.cppreference.com/w/c/types/size_t
https://en.cppreference.com/w/c/types/ptrdiff_t
https://en.cppreference.com/w/c/language/pointer

Effective Type

- Every object has an effective type.
- Variables have the effective type they were declared with.
- But the object that pointers point to may have a different effective type than the pointer says.
- We have already seen this with arrays:

```
int arr[10];
int *p = &arr[5];
int i = 42;
int *pp = &i;
```

- `arr` has effective type `int[10]`
- `p` and `pp` has effective type `int*`
- The pointee type of `p` and `pp` is `int`
- `p` points into an object which has effective type `int[10]`
- `pp` points to an object which has effective type `int`

---

https://en.cppreference.com/w/c/language/object

Effective Type and Strict Aliasing

- `void*` is a generic pointer that can point to objects of any effective type.
- But, we may only access objects through pointers that have a type which is "compatible" with the effective type.
- This is the "strict aliasing rule" (don't do nasty type tricks).
- Except, we may store/retrieve objects from character arrays.
- We can change the effective type through **`memcpy`**.

```
int i = 42;
int *ip = &i;
float *fp = (float*)ip;
float f = *fp; // boom, UB:
// fp points to an object with effective type int,
// but we try to dereference it as if it is a float,
// and int and float are not compatible types.
```

- Preliminaries

- Dynamic Memory Allocation

Allocation

`max_align_t`:

- Type alias for a type with the strictest alignment among fundamental types.
- Note: this may be too little when using SSE explicitly.

```
void *malloc(size_t size);
```

- Allocate a block of memory of size `size`.
- Returns `NULL` on failure.
- Otherwise returns a pointer to the first byte.
- The memory is uninitialised.
- The memory is aligned as `max_align_t`.
- It has no effective type, until we write to it (through something else than character types).

```
void *aligned_alloc(size_t alignment, size_t size);
```

- New in C11. As `malloc`, but with the specified alignment.

`https://en.cppreference.com/w/c/memory/malloc`

Examples

- Allocate memory for just 1 `double`:
  ```
  double *dp = malloc(sizeof(double));
  ```
- Allocate memory for 100 `int`s:
  ```
  int *arr = malloc(100 * sizeof(int));
  ```
- Same thing:
  ```
  int *arr = malloc(sizeof(int[100]));
  ```
- Allocate memory for a `Triangle`:
  (assuming we did `typedef struct Triangle Triangle;`)
  ```
  Triangle *t = malloc(sizeof(Triangle));
  ```

- Allocate memory suitable for `zmm` registers:
  ```
  double *dp = aligned_alloc(64, 64*42);
  ```

More Memory Management Functions

```
void free(void *ptr);
```

- Deallocate the memory pointed to.
- The given `ptr` must come from one of the allocation functions.
- Be careful not to use, or `free`, the memory again afterwards.

```
void *calloc(size_t num, size_t size);
```

- Allocate space for `num` objects, each of size `size`, and then zero-fill the memory.
- (Technically, zero-bytes doesn't guarantee that floats and pointers get value 0)

```
void *realloc(void *ptr, size_t new_size);
```

- As `malloc`, but copies the content of the old memory to the new.
- The given `ptr` must come from one of the allocation functions.
- The reallocation may happen in-place,
- but the old pointer may no longer be used, no matter what.

`https://en.cppreference.com/w/c/memory`

Example: A Doubly Linked List for `int`

- We want insertion in the front/back in constant time.
- We want to be able to query the size of the list in constant time.

Types:

```
4  // Public
5  // ================================================
6
7  typedef struct IntLinkedList IntLinkedList;
8  typedef struct IntLinkedListNode IntLinkedListNode;
9
10 struct IntLinkedList {
11    IntLinkedListNode *head, *tail;
12    int size;
13 };
14
15 // Private
16 // ================================================
17
18 struct IntLinkedListNode {
19    int data;
20    IntLinkedListNode *prev, *next;
21 };
```

Linked List Operations

- C doesn't directly support object-oriented programming.
- But we can approximate it to some extend.

"Public":

- Allocate and construct (initialise) an `IntLinkedList`.
- Destruct a `IntLinkedList` (deallocate all nodes).
- Insert an element in the back.
- Remove an element.
- . . . (more linked list operations)

"Private":

- Allocate and initialise a `LinkedListNode`.

Linked List Operations

(see example code)